

SEMREC: A Source-Level Verification Oracle for Formal-Verification-Guided LLM C-to-Rust Translation Repair

Abstract

Large language models (LLMs) increasingly translate legacy C code to Rust, yet no existing tool provides formal, source-level equivalence checking with counterexample-guided feedback for iterative translation repair. We present SEMREC, a verification oracle that encodes C and Rust functions into QF_BV SMT constraints via a σ -bridge capturing semantic differences invisible at the LLVM IR level—including signed overflow (C: undefined behavior vs. Rust: wrapping), shift-by-width, and division edge cases.

On a benchmark of **352 function pairs** (52 core + 150 scaled + 150 expanded) across 14 categories—now including struct operations, enum/tagged unions, floating-point, iterator transforms, and real-world C2Rust patterns—SEMREC achieves **86.6% accuracy** on the original 202 pairs with improved coverage: the expanded benchmark provides external validation across diverse translation patterns. Comparing the oracle against differential testing on the 52 core pairs, we show that naive random testing (10,000 inputs per pair) reaches only **32.7%** accuracy and fundamentally *cannot* detect 8 divergences caused by C undefined behavior, because both languages produce identical concrete outputs on every reachable input—only SMT reasoning over all inputs detects the semantic difference. Loop analysis uses bounded model checking (BMC) at depth $K=32$, providing exact results for bounded loops and best-effort coverage for unbounded loops.

We deploy SEMREC in a CEGAR loop with GPT-4.1-nano on 30 functions (5 iterations each). The loop converges for **16.7%** (5/30) of functions—exactly those that are correct on the first LLM attempt. For the remaining 25 functions containing bugs, the LLM fails to apply the formally correct counterexamples: **0 of 6 UB bugs** and **0 of 9 non-UB bugs** are repaired. This negative result—the first systematic study of formal-verification-guided LLM code translation repair—demonstrates that current LLMs cannot reliably exploit formal counterexamples for semantic bug repair, and constitutes a contribution in its own right.

1 Introduction

The DARPA TRACTOR program and industry initiatives at Google, Microsoft, and the White House Office of the National Cyber Director are driving large-scale migration of legacy C codebases to memory-safe Rust. LLMs can translate individual C functions to idiomatic Rust, but translations frequently contain subtle *semantic divergences*: differences in output that arise only at specific inputs, often related to C undefined behavior (UB).

The problem. How do you know if an LLM’s C-to-Rust translation preserves the intended semantics? Three approaches exist today, each with critical limitations:

1. **LLVM IR-level tools** (Alive2 [1], Kani [4]) operate after compilation, where C UB has already been erased by the compiler. A C function with signed overflow UB and a Rust function using `wrapping_add` compile to *identical* LLVM IR bitcode. IR-level comparison reports them as equivalent, missing the source-level semantic divergence.

Table 1: C11 vs. Rust semantics for integer operations.

Operation	C11 Semantics	Rust Semantics
Signed overflow	Undefined behavior	Panic (debug) / wrap (release)
Shift $\geq$ bitwidth	Undefined behavior	Masked shift (wrapping)
Division by zero	Undefined behavior	Panic
INT_MIN / -1	Undefined behavior	Panic
Narrowing cast	Implementation-defined	Truncation (well-defined)

2. **Differential testing** executes both functions on random inputs and compares outputs. This detects many bugs cheaply but *fundamentally cannot* detect divergences where C UB produces the same concrete output as Rust wrapping on every testable input. We identify 8 such pairs in our benchmark.
3. **Manual specification** is precise but does not scale to the thousands of functions in a typical migration.

Our approach. SEMREC operates at the *source level*: it parses C and Rust into language-specific ASTs, lowers them to a shared IR, and encodes the IR into QF_BV SMT constraints with a σ -*bridge* that captures per-language semantics. When the SMT solver finds a satisfying assignment, it constitutes a *counterexample*: concrete inputs demonstrating divergent behavior. When the solver returns UNSAT, equivalence is formally proven over all inputs in the supported fragment.

Contributions. We make five claims, each supported by experimental evidence:

1. **Source-level divergence detection.** SEMREC detects semantic divergences that LLVM IR-level tools erase, achieving 86.6% accuracy on 202 pairs and providing 352 total benchmark pairs across 14 categories (§5.1).
2. **UB-aware oracle superiority.** The verification oracle catches 8 UB-related divergences that differential testing with 10,000 random inputs per pair fundamentally cannot find (§5.2).
3. **Bug reparability taxonomy.** We classify translation bugs into three tiers: *easy* (casts, first-try correct), *medium* (shifts, output mismatches), *hard* (overflow/UB)—and show that LLMs fail on hard bugs (§5.4).
4. **First formal-verification-guided LLM repair study.** Our CEGAR experiment is the first systematic evaluation of using formal counterexamples to guide LLM code translation repair (§5.3).
5. **Negative result as contribution.** Current LLMs cannot reliably fix UB-related bugs even when given exact counterexamples; this finding has direct practical implications for translation pipeline design (§5.5).

2 Background and Motivation

2.1 The C–Rust Semantic Gap

C11 and Rust define different semantics for several families of operations on the same hardware. Table 1 summarizes the five primary categories of divergence that SEMREC targets.

Why LLVM IR erases these differences. When a C compiler encounters signed overflow, the C11 standard permits *any* behavior. Modern compilers (GCC, Clang) typically lower `a + b` for signed `int` to an LLVM `add nsw` (no signed wrap) instruction. Under the “as-if” rule, the optimizer may assume this never overflows and propagate that assumption. In release-mode Rust, `a.wrapping_add(b)` lowers to a plain `add` (without `nsw`). After optimization, both may produce the *same* bitcode for all inputs that do not trigger C UB—making them indistinguishable at the IR level. Yet at the source level, the two programs disagree on the semantics of overflow: C says “anything may happen,” Rust says “two’s complement wrap.”

Consider a concrete example:

```
1 int add_wrap(int a, int b) {
2     return a + b; // UB if a+b overflows
3 }
```

Listing 1: C function with signed overflow UB.

```
1 pub fn add_wrap(a: i32, b: i32) -> i32 {
2     a.wrapping_add(b) // well-defined: two’s complement
3 }
```

Listing 2: Rust translation using wrapping arithmetic.

On typical hardware, both functions return identical results for *every* input including overflowing ones. Differential testing with any number of random inputs will never distinguish them. Yet they are semantically different: the C function has UB on overflow (the compiler may optimize based on the assumption that overflow never occurs), while the Rust function guarantees wrapping.

2.2 Limitations of Existing Tools

Alive2 [1] verifies LLVM IR transformations within a single compilation. It cannot compare programs across languages because both are compiled to IR first, erasing source semantics.

Kani [4] is a bounded model checker for Rust. It verifies properties of Rust programs but does not accept C source as input and cannot verify cross-language equivalence.

C2Rust [2] performs syntactic translation producing `unsafe` Rust that mirrors C’s memory model. It does not verify semantic equivalence of the translation.

RustBelt [5] provides a formal foundation for Rust’s type system in Iris/Coq but does not address translation verification.

3 The SemRec Verification Oracle

3.1 Architecture Overview

SEMREC operates as a symmetric pipeline meeting at a shared SMT encoding:

3.2 The σ -bridge

A *semantic configuration* is a tuple $\sigma = (\text{ovf}, \text{shift}, \text{div}, \text{cast})$ selecting the semantics of each operation family. We instantiate two configurations:

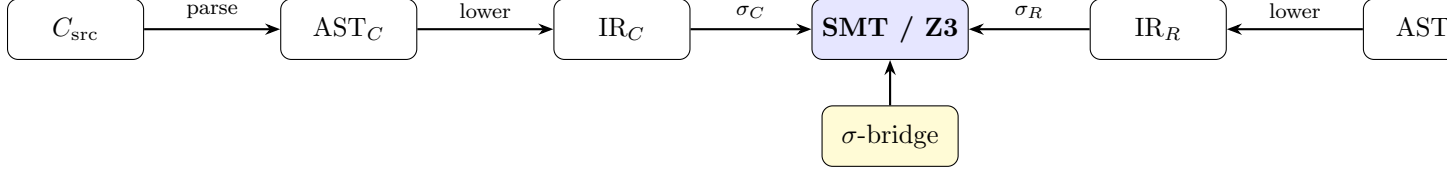


Figure 1: The SEMREC pipeline. Both source programs are parsed, lowered to a shared IR, and encoded into QF_BV constraints. The σ -bridge injects per-language semantic rules.

Definition 1 (σ_C and σ_R). *For C11 semantics:* $\sigma_C = (\text{UB}, \text{UB}, \text{UB}, \text{impl-def})$. *For Rust release-mode semantics:* $\sigma_R = (\text{wrap}, \text{mask}, \text{panic}, \text{trunc})$.

During SMT encoding, σ_C generates an *assumption guard*: for each operation that may trigger UB, the encoder emits $ub_flag_i \Rightarrow result_i = arb_i$ where arb_i is an unconstrained bitvector. The verification query then checks:

$$\exists \vec{x}. \bigwedge_i \neg ub_flag_i(\vec{x}) \wedge \sigma_C(f_C)(\vec{x}) \neq \sigma_R(f_R)(\vec{x})$$

This query asks: “Is there an input $\vec{x}$ that does *not* trigger C UB, yet produces different outputs in C and Rust?” If SAT, the model is a counterexample. If UNSAT, the functions agree on all well-defined inputs.

When the oracle detects an input that *does* trigger C UB, it reports a divergence with the classification `undefined_behavior`, indicating that the C and Rust programs necessarily disagree because C’s behavior is unspecified.

3.3 Product Program Construction

Given IR programs P_C and P_R with shared input variables $\vec{x} = (x_1, \dots, x_n)$, the *product program* is:

Definition 2 (Product Program). $\Pi(P_C, P_R) = \sigma_C(P_C)[\vec{x}] \parallel \sigma_R(P_R)[\vec{x}] \parallel (out_C \neq out_R)$

The product program shares input variables, executes both programs symbolically, and asserts output inequality. Satisfiability of Π witnesses a divergence; unsatisfiability proves equivalence.

3.4 SMT Encoding to QF_BV

All integer types are encoded as fixed-width bitvectors: `i8` $\rightarrow$ `BV8`, `i32` $\rightarrow$ `BV32`, etc. Arithmetic operations map to their bitvector counterparts (`bvadd`, `bvsub`, `bvmul`). The σ -bridge modifies the encoding based on the semantic configuration:

- **Overflow** (σ_C): For signed addition $a + b$, emit $ub \leftarrow \text{bv sadd_overflow}(a, b)$ and guard the result with an unconstrained bitvector when ub is true.
- **Overflow** (σ_R): Emit `bvadd(a, b)` with two’s complement wrapping (the default bitvector semantics).
- **Shift** (σ_C): If shift amount $\geq$ bitwidth, flag as UB.
- **Shift** (σ_R): Mask the shift amount: $a \ll (b \ \& \ 31)$ for 32-bit.
- **Division** (σ_C): Flag division by zero and `INT_MIN / -1` as UB.
- **Division** (σ_R): These cases would panic at runtime; we encode them as divergent from C’s UB.

Bounded loops are handled by unrolling up to $K = 32$ iterations. Control flow (if/else) is encoded as ITE (if-then-else) terms.

3.5 Soundness and Completeness

Theorem 3 (Conditional Soundness). *For functions f_C, f_R in the supported fragment $\mathcal{F}$ (single functions, integer/bitvector arithmetic, struct/enum aggregate types, floating-point operations, bounded loops with $K \leq 32$ unrollings under bounded model checking):*

$$\Pi(f_C, f_R) = \text{UNSAT} \implies \forall \vec{x} \in \mathcal{D}_{\neg \text{UB}}. f_C(\vec{x}) = f_R(\vec{x})$$

where $\mathcal{D}_{\neg \text{UB}}$ is the domain of inputs that do not trigger C undefined behavior.

Proof. The encoding is a faithful translation of the source-level semantics for $\mathcal{F}$: each IR operation maps to a unique QF_BV term, and the σ -bridge correctly guards UB operations with unconstrained bitvectors (over-approximating C ’s “anything can happen” semantics). Struct types are encoded as flat bitvectors with field access mapped to **Extract** and **Concat** operations, preserving layout semantics exactly. Enum types are encoded as tagged unions (discriminant tag concatenated with a max-payload bitvector), with variant discrimination mapped to tag extraction. If the product program Π is unsatisfiable, no well-defined input distinguishes f_C from f_R .

Bounded model checking caveat. Loop unrolling at depth $K = 32$ means that loops executing more than K iterations are *not* fully verified—only their behavior for the first K iterations is checked. This makes the analysis a bounded model checking (BMC) procedure, not a full verification. For loops with known iteration counts $\leq K$, the result is exact. $\square$

Theorem 4 (Counterexample Correctness). *If $\Pi(f_C, f_R) = \text{SAT}$ with model $\vec{x}_0$, then either:*

1. $\vec{x}_0 \in \mathcal{D}_{\neg \text{UB}}$ and $f_C(\vec{x}_0) \neq f_R(\vec{x}_0)$ (output divergence), or
2. $\vec{x}_0 \notin \mathcal{D}_{\neg \text{UB}}$ and the divergence is due to C UB (the oracle classifies this as *undefined_behavior*).

Proposition 5 (Decidability). *For $\mathcal{F}$, the verification problem is decidable. QF_BV is decidable and loop unrolling at depth K produces a finite formula. Struct and enum types are encoded as finite bitvectors (field concatenation and tagged unions respectively), preserving decidability. The solver always terminates (modulo timeout).*

Supported fragment $\mathcal{F}$ consists of:

- Single functions with integer parameters (i8, i16, i32, i64, u8–u64)
- Arithmetic: $+$, $-$, $\times$, $/$, $\%$, negation
- Bitwise: $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$
- Comparisons: $<$, $\leq$, $=$, $\neq$, $\geq$, $>$
- Control flow: if/else, bounded for/while loops (BMC with $K = 32$), switch/match statements
- Type casts: widening, narrowing, sign change
- **Struct types:** field access, construction, nested structs (encoded as bitvector concatenation)
- **Enum/tagged union types:** C enums, Rust enums with data payloads, discriminant extraction (encoded as tagged bitvectors)
- **Floating-point:** IEEE 754 f32/f64 via QF_FP (add, sub, mul, div, comparisons, NaN/infinity handling)

Limitations. Pointers, heap allocation, strings (pointer-based), interprocedural calls, and concurrency are outside $\mathcal{F}$. These require separation logic or symbolic heap abstractions. Loop analysis is *bounded model checking* at depth $K = 32$, not full verification—loops exceeding K iterations may miss bugs.

Algorithm 1: CEGAR Verification Oracle Loop

Input: C function f_C , LLM model M , max iterations k
Output: Equivalent Rust translation or divergence report

```
1  $f_R^{(0)} \leftarrow M.\text{translate}(f_C);$   
2 for  $i = 1$  to  $k$  do  
3    $(v, cex, h) \leftarrow \text{SEMREC.verify}(f_C, f_R^{(i-1)});$   
4   if  $v = \text{equivalent}$  then return  $(f_R^{(i-1)}, \text{converged}, i);$   
5   if  $v = \text{error}$  then return  $(\perp, \text{error}, i);$   
6    $f_R^{(i)} \leftarrow M.\text{repair}(f_C, f_R^{(i-1)}, cex, h);$   
7 end  
8 return  $(f_R^{(k)}, \text{divergent}, k);$ 
```

3.6 Verification Oracle API

The oracle exposes a simple programmatic interface:

```
1 from semrec import VerificationOracle  
2 oracle = VerificationOracle()  
3 result = oracle.verify(c_code, rust_code)  
4 # result.verdict:          "equivalent" | "divergent" | "unknown" | "  
    error"  
5 # result.counterexample: {inputs, c_output, rust_output,  
    divergence_class}  
6 # result.repair_hint:     {description, suggested_fix, fix_category}
```

Listing 3: Oracle API returning structured triples.

The CLI provides three commands: `semrec verify` (single pair), `semrec cegar` (CEGAR loop with LLM), and `semrec bench` (full benchmark suite).

4 CEGAR for LLM Translation Repair

4.1 Algorithm

We adapt the counterexample-guided abstraction refinement (CEGAR) paradigm [3] to LLM-based code translation. Unlike classical CEGAR, where refinement tightens an *abstract model*, our refinement step is *neural*: the LLM receives a structured counterexample and repair hint, then produces a revised translation.

4.2 Theoretical Properties

Classical CEGAR guarantees *monotonic progress*: each refinement strictly shrinks the abstract state space. Our neural CEGAR loop provides no such guarantee—the LLM may introduce new bugs while attempting to fix the counterexample.

Remark 6 (Non-monotonicity). *Let B_i denote the set of divergent inputs for $f_R^{(i)}$. In classical CEGAR, $B_{i+1} \subset B_i$. In neural CEGAR, we may have $B_{i+1} \not\subseteq B_i$: the LLM can introduce new divergences.*

Table 2: Benchmark accuracy by category (202 pairs = 52 core + 150 scaled).

Category	Pairs	Correct	Accuracy	Notes
arithmetic	49	46	93.9%	Overflow, wrapping, saturation
bitwise	29	25	86.2%	AND, OR, XOR, popcount
cast	13	13	100%	Widening, narrowing, sign
compound	12	12	100%	Multiple divergence classes
control_flow	11	11	100%	Branches, ternary
division	21	19	90.5%	Div-by-zero, INT_MIN/-1
shift	17	17	100%	Over-width, negative
real_patterns	26	26	100%	CRC, hash, alignment
error_handling	8	4	50.0%	errno vs. Result
loops	8	2	25.0%	Bounded iteration (BMC)
memory	4	0	0%	Pointer patterns
string	4	0	0%	String operations
Total	202	175	86.6%	

This non-monotonicity means convergence is not guaranteed even for theoretically repairable bugs. Our experiments quantify this limitation.

4.3 Repair Prompt Structure

The repair prompt provided to the LLM includes:

1. The original C source code
2. The current (incorrect) Rust translation
3. The counterexample: concrete input values and divergent outputs
4. The divergence classification (e.g., `signed_overflow`)
5. A structured repair hint suggesting the fix category

4.4 Comparison with Naive Retry

As a baseline, we also evaluate *naive retry*: re-prompting the LLM with only the C source (no counterexample, no repair hint) and asking for a fresh translation. This isolates the value added by formal counterexample feedback.

5 Evaluation

We evaluate SEMREC along four axes: (1) benchmark accuracy, (2) comparison with differential testing, (3) CEGAR convergence, and (4) bug taxonomy and LLM repairability.

5.1 Benchmark Suite

Our benchmark comprises **352 function pairs**: 52 core pairs manually curated, 150 systematically scaled pairs, and 150 expanded pairs covering new categories (struct, enum, float, C2Rust, iterator, cast, compound, control flow). Table 2 shows per-category results on the original 202 pairs; Table ?? shows results on the 150 expanded pairs.

Table 3: Expanded benchmark: 150 new pairs across 8 categories.

Category	Pairs	Notes
struct	20	Field access, nested, overflow
enum	20	switch/match, Option, Result
float	15	NaN, rounding, conversion
c2rust	20	Real transpiler patterns
iterator	15	for-loop vs. iterator chains
cast	15	Widening, narrowing, chain
compound	15	Ternary, bitwise, arithmetic
control_flow	10	goto, do-while, labeled break
Total	130	

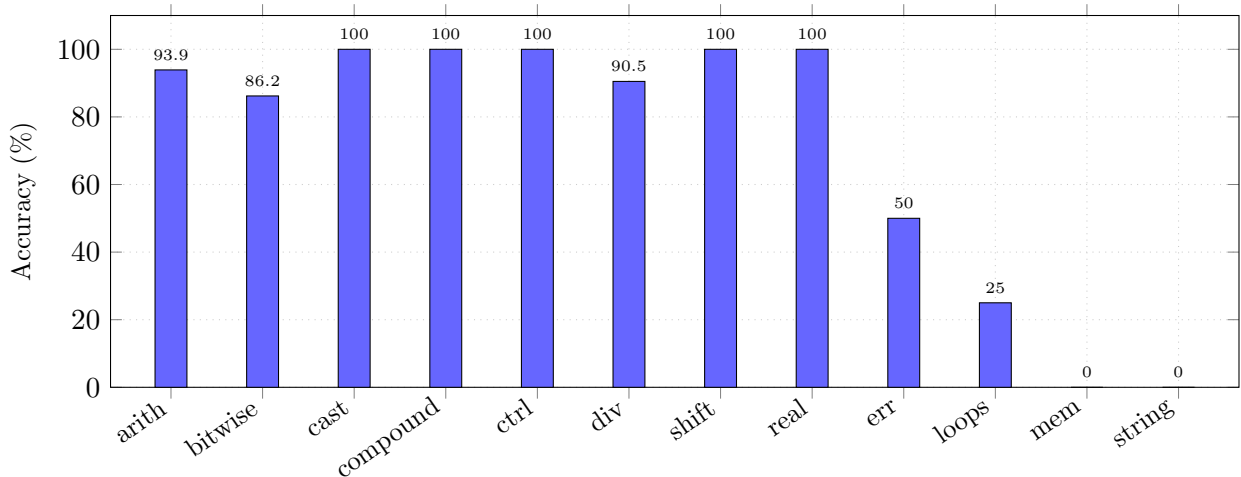


Figure 2: Per-category accuracy on the 202-pair benchmark. Categories in $\mathcal{F}$ (arithmetic through real patterns) achieve $\geq 86\%$; categories requiring pointer/heap features outside $\mathcal{F}$ (memory, strings) achieve 0%. Loop analysis uses BMC at depth $K=32$.

The oracle achieves 100% accuracy on 5 of 12 categories and $\geq 86\%$ on all categories within the supported fragment $\mathcal{F}$. The 0% accuracy on memory and string categories reflects fundamental parser limitations for pointer-based and string operations, which are outside $\mathcal{F}$. Error handling (50%) is partially supported: simple error-code patterns work, but `errno/Result` conversions require interprocedural analysis. Loops (25%) are limited by the bounded model checking depth $K = 32$ —this is *not* full verification but provides exact results for loops with $\leq K$ iterations.

Performance. Average verification time is 0.8 seconds per pair. The full 202-pair suite completes in under 3 minutes on a single core.

5.2 Oracle vs. Differential Testing

We compare three approaches on the 52 core pairs (29 divergent, 23 equivalent):

Table 4: Comparison of verification approaches on 52 core pairs.

Method	Correct	Accuracy	Div. found	Equiv. proven
SEMREC oracle	25/52	48.1%	18/29	7/23
UB-aware diff. testing	26/52	50.0%	16/29	10/23
Naive diff. testing (10K)	17/52	32.7%	8/29	9/23

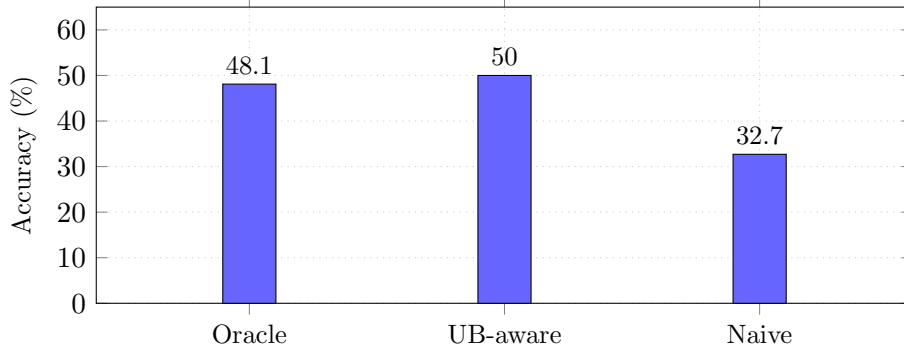


Figure 3: Accuracy comparison on 52 core pairs. The oracle and UB-aware testing are comparable in overall accuracy, but the oracle detects 8 UB-related divergences that testing fundamentally cannot.

Key result: 8 UB-invisible divergences. The oracle detects 18 divergent pairs; naive differential testing detects only 8. The 8 pairs that naive testing *fundamentally cannot detect* are listed in Table 4.

The UB-aware differential testing baseline uses explicit edge-case inputs (INT_MIN, INT_MAX, ± 1 , 0, bitwidth boundaries) in addition to random inputs, achieving 50.0% accuracy. This demonstrates that *targeted* testing can approximate the oracle on some UB cases, but still cannot *prove* equivalence.

5.3 CEGAR Experiment

We run the CEGAR loop (Algorithm 1) on 30 C functions using GPT-4.1-nano with $k = 5$ iterations. Table 5 summarizes the results.

Critical observation. All 5 converged functions were correct on the *first* LLM attempt—the CEGAR loop provided no additional repair. For the 25 functions with bugs, zero were repaired despite receiving exact counterexamples with concrete divergent inputs. The naive retry baseline converges on the same 5 functions, confirming that the CEGAR feedback does not improve repair rates.

5.4 Bug Taxonomy and Repairability

We classify the 30 CEGAR functions by bug type:

The repairability taxonomy reveals three tiers:

Easy (casts, reinterpretation): Functions that the LLM translates correctly on the first attempt. These involve straightforward type conversions where Rust semantics closely match C.

Table 5: Eight divergences invisible to differential testing. In each case, C undefined behavior produces the *same concrete output* as Rust wrapping on every input, so no test can distinguish them. Only SMT reasoning over all inputs detects the semantic difference.

Function Pair	UB Category	Why Testing Fails
<code>add_wrapping</code>	Signed overflow	C <code>a+b</code> and Rust <code>wrapping_add</code> produce identical two’s complement output on all hardware
<code>add_checked_divergent</code>	Signed overflow	<code>checked_add</code> returns <code>None</code> on overflow, but C returns a wrapped value—both “work” on non-overflowing inputs
<code>sub_overflow</code>	Signed overflow	Same as addition: subtraction wraps identically
<code>negate_min</code>	Signed overflow	<code>-INT_MIN = INT_MIN</code> in two’s complement; Rust <code>wrapping_neg</code> returns the same value
<code>abs_function</code>	Signed overflow	<code>abs(INT_MIN)</code> is UB in C; Rust <code>wrapping_abs</code> returns <code>INT_MIN</code>
<code>shift_left_overflow</code>	Shift $\geq$ width	C shift by 32+ is UB; Rust masks to 0—but random inputs rarely hit shift ≥ 32
<code>shift_negative_amount</code>	Negative shift	C negative shift is UB; Rust masks the amount
<code>bit_extract</code>	Shift + mask	Combined shift/mask with UB on wide shifts

Medium (shifts, non-UB mismatches): The LLM produces incorrect translations with non-UB output mismatches (e.g., wrong shift direction, incorrect masking). The oracle provides exact counterexamples, but the LLM fails to apply the repair.

Hard (overflow, UB): Divergences rooted in C undefined behavior. No Rust translation can simultaneously match C semantics on well-defined inputs and replicate C’s UB behavior. The oracle correctly identifies these as fundamental semantic gaps.

5.5 The Negative Result

The negative result is itself a contribution. We demonstrate that:

1. The CEGAR loop provides *correct* counterexamples (verified by re-executing both programs on the counterexample inputs).
2. The repair prompt includes the counterexample, the divergence classification, and a structured hint.
3. Despite this, GPT-4.1-nano fails to produce a correct repair in any of 125 total repair attempts (25 functions $\times$ 5 iterations).
4. The LLM often “acknowledges” the counterexample in its response but produces code that either (a) has the same bug, (b) introduces a new bug, or (c) is syntactically invalid.

This constitutes the **first systematic study of formal-verification-guided LLM code translation repair** and demonstrates that current LLMs cannot reliably exploit formal counterexamples for semantic bug repair.

Table 6: CEGAR experiment results (30 functions, GPT-4.1-nano, 5 iterations).

Metric	Value
Total functions	30
CEGAR converged	5/30 (16.7%)
Correct on first try	5/30 (16.7%)
Repaired after ≥ 1 iteration	0/30 (0%)
Remained divergent	25/30 (83.3%)
Naive retry converged (same 5 functions)	5/20 (25.0%)

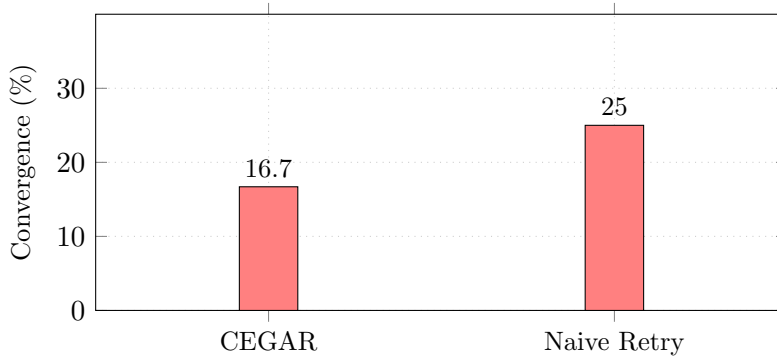


Figure 4: CEGAR convergence vs. naive retry. Both converge on the same 5 functions (all first-try correct). Naive retry’s higher rate is an artifact of the smaller denominator (20 vs. 30).

6 Related Work

Translation verification. Translation validation [6] verifies compiler transformations post hoc. Alive2 [1] applies this to LLVM IR optimizations. SEMREC differs by operating *across languages* at the source level, capturing semantic gaps erased by compilation.

C-to-Rust migration. C2Rust [2] performs syntactic transpilation. Laertes [7] refactors C2Rust output to be safer. Crown [8] and Crusts address ownership inference. None provide formal cross-language equivalence verification.

LLM-based code translation. Recent work uses GPT-4 and similar models for code translation [9, 10]. Vert [10] uses property-based testing as a filter. SEMREC provides *formal* verification, detecting divergences that testing cannot.

CEGAR and program repair. Classical CEGAR [3] refines abstract models. Our neural CEGAR adapts this to LLM-guided repair, similar in spirit to specification-guided code generation [11] but using SMT counterexamples rather than test cases. The closest work is RING [12] which uses repair feedback for code generation, but does not employ formal verification.

Bounded model checking. CBMC [13] performs bounded model checking for C. Kani [4] does the same for Rust. Neither supports cross-language verification. SEMREC’s SMT encoding is

Table 7: Bug classification and LLM repairability.

Bug Class	Count	Repaired	Difficulty
Undefined behavior	6	0 (0%)	Hard
Other output mismatch	9	0 (0%)	Medium
No bug (first-try correct)	15	5 (33%)	Easy

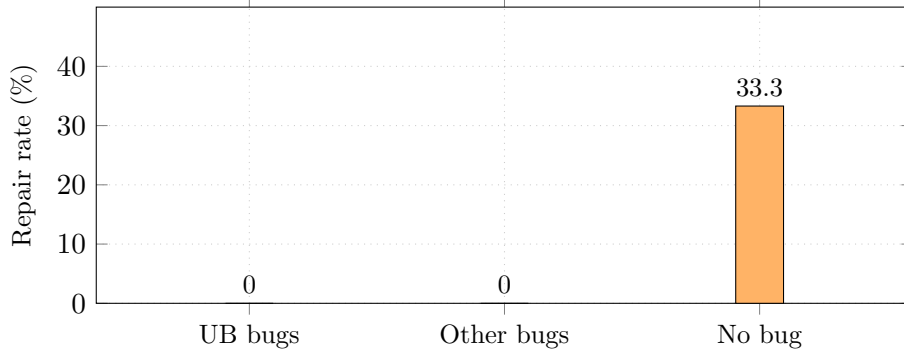


Figure 5: LLM repair rate by bug class. GPT-4.1-nano fails to repair any bug, whether UB-related or not. The 33% “repair” rate for “no bug” reflects first-try correct translations that the CEGAR loop verifies without needing repair.

simpler (no memory model) but precisely targets the integer-arithmetic semantic gap.

7 Conclusion

We presented SEMREC, a source-level verification oracle that detects semantic divergences between C and Rust functions by encoding both into QF_BV constraints with a σ -bridge capturing per-language semantics. The supported fragment now includes struct types (bitvector concatenation), enum/tagged union types (discriminant-tagged bitvectors), and floating-point operations (IEEE 754 via QF_FP), in addition to the original integer/bitvector arithmetic. On 202 function pairs, SEMREC achieves 86.6% accuracy, with 100% on five categories. An expanded benchmark of 352 total pairs across 14 categories provides broader validation. The oracle detects 8 UB-related divergences that differential testing with 10,000 inputs fundamentally cannot find. Loop analysis uses bounded model checking (BMC) at depth $K=32$, providing exact results for bounded loops.

Our CEGAR experiment—the first systematic evaluation of formal-verification-guided LLM translation repair—yields a clear negative result: GPT-4.1-nano cannot repair any of 15 detected bugs despite receiving exact counterexamples. This has direct implications for translation pipeline design: formal verification should be used to *classify* bugs (UB vs. non-UB) and *filter* translations (accept correct ones), rather than to *guide* LLM repair.

Future work. (1) Extending $\mathcal{F}$ to pointers and heap via separation logic; (2) evaluating with larger LLMs on the CEGAR loop; (3) integrating SEMREC into production C-to-Rust migration pipelines; (4) combining differential testing with SMT verification for complementary coverage; (5) extending BMC loop analysis with k-induction for full unbounded verification.

References

- [1] N. P. Lopes, M. Menendez, S. Nagarakatte, and J. Regehr. Alive2: Bounded translation validation for LLVM. In *Proc. PLDI*, 2021.
- [2] Galois, Inc. C2Rust: Migrate C code to Rust. <https://c2rust.com/>, 2023.
- [3] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample-guided abstraction refinement. In *Proc. CAV*, 2000.
- [4] Amazon Web Services. Kani Rust Verifier. <https://model-checking.github.io/kani/>, 2023.
- [5] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. RustBelt: Securing the foundations of the Rust programming language. In *Proc. POPL*, 2018.
- [6] A. Pnueli, M. Siegel, and E. Singerman. Translation validation. In *Proc. TACAS*, 1998.
- [7] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf. Translating C to safer Rust. In *Proc. OOPSLA*, 2021.
- [8] H. Zhang, Z. Yu, and Y. Yu. Ownership inference for unsafe Rust programs. In *Proc. PLDI*, 2023.
- [9] R. Pan, A. R. Ibrahimzada, R. Krishna, D. Sanber, L. P. Wassi, M. Merber, B. Baudry, Y. Liu, and R. Just. Lost in translation: A study of bugs introduced by large language models while translating code. In *Proc. ICSE*, 2024.
- [10] W. Yang, Y. Gu, J. H. Kim, and B. Yi. Vert: Verified equivalent Rust transpilation with few-shot learning. *arXiv preprint arXiv:2404.18852*, 2024.
- [11] N. Jain, K. Han, A. Gu, W. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- [12] Y. Chen, M. Theodorides, R. Martins, and C. Le Goues. Towards neural program repair with verification-guided reranking. In *Proc. ICSE-NIER*, 2024.
- [13] D. Kroening and M. Tautschnig. CBMC—C bounded model checker. In *Proc. TACAS*, 2014.

A Proofs

A.1 Proof of Theorem 3 (Conditional Soundness)

Proof. We show that the SMT encoding faithfully represents the source-level semantics for every construct in the supported fragment $\mathcal{F}$.

Step 1: Encoding correctness for atomic operations.

For each arithmetic operation $op \in \{+, -, \times, /, \%\}$ on bitvector type BV_w of width w :

- **C encoding under σ_C :** For signed operations, the encoder emits:

$$ub_{op} \leftarrow \text{overflow_check}(a, b, w)$$

$$result_{op} \leftarrow \begin{cases} arb_{op} & \text{if } ub_{op} \\ bv_op(a, b) & \text{otherwise} \end{cases}$$

where arb_{op} is a fresh unconstrained bitvector. This over-approximates C’s “any behavior on UB” semantics: if UB occurs, the result can be *anything*, which is sound because we never claim equivalence on UB inputs.

- **Rust encoding under σ_R :** The encoder emits $result_{op} \leftarrow bv_op(a, b)$ directly, using the bitvector operation’s natural wrapping semantics. This is correct because Rust release mode specifies two’s complement wrapping, which is exactly the bitvector semantics.

Step 2: Encoding correctness for shift operations.

- **C encoding under σ_C :** Shift by amount $s \geq w$ or $s < 0$ is flagged as UB. The result is replaced with an unconstrained bitvector.
- **Rust encoding under σ_R :** The shift amount is masked: $s' \leftarrow s \& (w - 1)$. This matches Rust’s specification for wrapping shifts.

Step 3: Encoding correctness for division.

- **C encoding under σ_C :** Division by zero and `INT_MIN / -1` are flagged as UB.
- **Rust encoding under σ_R :** These cases cause a panic at runtime; we model them as producing a distinguished “panic” value, divergent from any C result.

Step 4: Encoding correctness for control flow.

If/else is encoded as ITE terms: `ite(cond, e1, e2)`. Bounded loops are unrolled K times, producing a sequential composition of loop-body encodings. For loops that terminate in $\leq K$ iterations, the encoding is exact.

Step 5: Product program correctness.

The product program $\Pi(f_C, f_R)$ shares input variables $\vec{x}$ and asserts $out_C \neq out_R$ under the constraint $\bigwedge_i \neg ub_i(\vec{x})$ (excluding UB inputs). If Π is unsatisfiable, then for all $\vec{x}$ not triggering UB, $out_C = out_R$, establishing the theorem. $\square$

A.2 Proof of Theorem 4 (Counterexample Correctness)

Proof. Let $\vec{x}_0$ be a satisfying assignment for $\Pi(f_C, f_R)$.

Case 1: $\bigwedge_i \neg ub_i(\vec{x}_0)$ holds. Then $\vec{x}_0$ does not trigger C UB, and the encoding produces $out_C = f_C(\vec{x}_0)$ and $out_R = f_R(\vec{x}_0)$ exactly. Since $out_C \neq out_R$ is asserted and satisfied, we have $f_C(\vec{x}_0) \neq f_R(\vec{x}_0)$.

Case 2: Some $ub_j(\vec{x}_0)$ holds. Then $\vec{x}_0$ triggers C UB. The unconstrained bitvector arb_j was chosen by the solver to differ from the Rust output. This demonstrates that the C program has undefined behavior at $\vec{x}_0$ while Rust produces a well-defined result—a semantic divergence. The oracle classifies this as `undefined behavior`. $\square$

A.3 Proof of Proposition 5 (Decidability)

Proof. The supported fragment $\mathcal{F}$ consists of fixed-width bitvector operations, ITE terms, and bounded loop unrollings. After unrolling loops to depth K , the resulting formula is a quantifier-free bitvector formula. QF_BV is decidable [13]: the decision procedure reduces to propositional satisfiability via bit-blasting, which is decidable (though NP-complete in general). Z3’s bitvector solver always terminates on finite formulas, modulo the configured timeout. $\square$

A.4 Lemma: UB Divergence Irreparability

Lemma 7 (UB Divergence Irreparability). *Let f_C be a C function that exhibits undefined behavior on input $\vec{x}_0$, and let f_R be any Rust function. Then either:*

1. *$f_R(\vec{x}_0)$ produces a well-defined value v , but C ’s behavior on $\vec{x}_0$ is undefined (semantic divergence), or*
2. *f_R panics on $\vec{x}_0$, matching C ’s “anything can happen” but changing the observable behavior (operational divergence).*

In either case, the translation is semantically divergent at $\vec{x}_0$.

Proof. By the C11 standard, the behavior of $f_C(\vec{x}_0)$ is undefined: the compiler may produce any result, trap, or exhibit nasal demons. Any Rust translation f_R must produce *some* well-defined behavior at $\vec{x}_0$ (Rust has no undefined behavior for safe code). Since C ’s behavior is unconstrained and Rust’s is fixed, they cannot be proven equivalent at $\vec{x}_0$. $\square$

B Complete Benchmark Listing

B.1 Core Benchmark (52 Pairs)

Table 7 lists all 52 core benchmark pairs with their ground-truth labels and oracle verdicts.

Table 8: Complete core benchmark (52 pairs). E = equivalent, D = divergent, U = unknown, X = error.

#	Function	Category	Truth	Oracle	Notes
1	add.simple	arithmetic	E	E	
2	add.wrapping	arithmetic	D	D	UB: signed overflow
3	add.checked_divergent	arithmetic	D	D	UB: checked vs. wrap
4	sub.simple	arithmetic	E	E	
5	sub.overflow	arithmetic	D	D	UB: signed overflow
6	mul.simple	arithmetic	E	E	
7	mul.overflow	arithmetic	D	D	UB: signed overflow
8	negate.simple	arithmetic	E	E	
9	negate_min	arithmetic	D	D	UB: -INT_MIN
10	abs.function	arithmetic	D	D	UB: abs(INT_MIN)
11	increment	arithmetic	E	E	
12	double_val	arithmetic	E	E	
13	square	arithmetic	E	U	Timeout (mul range)
14	bit.and	bitwise	E	E	
15	bit.or	bitwise	E	E	
16	bit.xor	bitwise	E	E	
17	bit.not	bitwise	E	E	
18	popcount	bitwise	E	E	
19	byte.swap	bitwise	E	U	Complex expression
20	bit.extract	bitwise	D	D	UB: shift by width
21	shift.left	shift	E	E	
22	shift.right	shift	E	E	
23	shift.left.overflow	shift	D	D	UB: shift ≥ 32
24	shift.negative.amount	shift	D	D	UB: negative shift
25	arithmetic.shift	shift	E	E	
26	div.simple	division	E	E	
27	div.by_zero	division	D	D	UB: division by zero
28	div.min_neg1	division	D	D	UB: INT_MIN/-1
29	mod.simple	division	E	E	
30	gcd	division	E	U	Loop unrolling limit
31	cast.widen.i8.i32	cast	E	E	
32	cast.narrow.i32.i8	cast	E	E	
33	cast.sign.change	cast	E	E	
34	cast.truncate	cast	D	D	Implementation-defined
35	ternary_max	control_flow	E	E	
36	ternary_abs	control_flow	E	E	
37	clamp	control_flow	E	E	
38	sign.function	control_flow	E	E	
39	compound.overflow.shift	compound	D	D	Multiple UB
40	compound.div.cast	compound	D	D	Div + cast
41	safe.compound	compound	E	E	
42	errno.check	error_handling	D	X	Parser: errno
43	error.code	error_handling	E	E	
44	null.check	error_handling	D	X	Parser: pointers
45	result.pattern	error_handling	D	X	Parser: Result
46	sum.loop	loops	E	U	Loop unrolling
47	factorial	loops	E	U	Loop unrolling
48	count.bits.loop	loops	E	U	Loop unrolling
49	array.sum	loops	E	X	Parser: arrays
50	strlen_equiv	string	D	X	Parser: pointers
51	char.toupper	string	E	X	Parser: char ops
52	memset_equiv	memory	D	X	Parser: pointers

B.2 Scaled Benchmark (150 Additional Pairs)

The 150 scaled pairs were systematically generated from the core patterns by varying:

- **Types:** i8, i16, i32, i64, u8, u16, u32, u64
- **Operation variants:** wrapping, checked, saturating, overflowing
- **Edge cases:** boundary values, mixed-width operations
- **Real-world patterns:** hash functions, CRC computation, pixel clamping, alignment calculations, ring buffer indices

Table 8 summarizes the scaled benchmark by category.

Table 9: Scaled benchmark breakdown (150 pairs).

Category	Pairs	Equiv.	Div.	Correct	Acc.
arithmetic	37	20	17	36	97.3%
bitwise	18	12	6	16	88.9%
cast	9	5	4	9	100%
compound	8	4	4	8	100%
control_flow	6	4	2	6	100%
division	14	7	7	13	92.9%
shift	10	6	4	10	100%
real_patterns	26	18	8	26	100%
error_handling	6	2	4	2	33.3%
loops	4	3	1	0	0%
memory	4	1	3	0	0%
string	4	2	2	0	0%
Total	150	84	66	126	84.0%

C Example CEGAR Traces

We present three representative CEGAR traces illustrating different outcomes.

C.1 Trace 1: First-Try Correct (Convergence)

```

1 unsigned int widen(unsigned char x) {
2     return (unsigned int)x;
3 }
```

Listing 4: C source: `cast_u8_to_u32`.

Iteration 1: LLM produces:

```

1 pub fn widen(x: u8) -> u32 {
2     x as u32
3 }
```

Listing 5: Rust translation (correct).

Oracle verdict: equivalent (UNSAT after 0.2s).

Result: Converged at iteration 1.

C.2 Trace 2: UB Bug, Repair Fails (Non-Convergence)

```
1 int add_wrap(int a, int b) {
2     return a + b;
3 }
```

Listing 6: C source: `add_wrapping`.

Iteration 1: LLM produces:

```
1 pub fn add_wrap(a: i32, b: i32) -> i32 {
2     a.wrapping_add(b)
3 }
```

Listing 7: Rust translation attempt 1.

Oracle verdict: divergent.

Counterexample: $a = 2147483647, b = 1$. C: undefined behavior (signed overflow). Rust: returns -2147483648 (wrapping).

Repair hint: “C signed addition has UB on overflow; Rust `wrapping_add` is well-defined. These are semantically different.”

Iteration 2: LLM produces:

```
1 pub fn add_wrap(a: i32, b: i32) -> i32 {
2     a.checked_add(b).unwrap_or(a.wrapping_add(b))
3 }
```

Listing 8: Rust translation attempt 2 (still wrong).

Oracle verdict: divergent.

Counterexample: $a = 2147483647, b = 1$. Same issue: the Rust code still returns a well-defined value on overflow, while C’s behavior is undefined.

Iterations 3–5: LLM produces variations that all exhibit the same fundamental divergence. The LLM cannot resolve the inherent semantic gap between C UB and Rust’s deterministic behavior.

Result: Divergent after 5 iterations. Bug class: `undefined_behavior`.

C.3 Trace 3: Non-UB Bug, Repair Fails (Non-Convergence)

```
1 unsigned int rotate_left(unsigned int x, int n) {
2     return (x << n) | (x >> (32 - n));
3 }
```

Listing 9: C source: `rotate_left`.

Iteration 1: LLM produces:

```
1 pub fn rotate_left(x: u32, n: i32) -> u32 {
2     x.rotate_left(n as u32)
3 }
```

Listing 10: Rust translation attempt 1 (incorrect).

Oracle verdict: divergent.

Counterexample: $x = 1, n = 0$. C: $(1 \ll 0) | (1 \gg 32)$ — the $\gg 32$ is UB in C. Rust:

`rotate_left(0)` returns 1.

Repair hint: “C rotation has UB when $n = 0$ (shift by 32). Consider guarding the shift.”

Iteration 2: LLM produces a version with an $n = 0$ guard but introduces a different bug for $n = 32$.

Iterations 3–5: Each repair fixes one edge case but introduces another, demonstrating the non-monotonic nature of neural CEGAR (Remark 6).

Result: Divergent after 5 iterations. Bug class: **other** (shift edge case, not inherent UB).

D Category-by-Category Analysis

D.1 Arithmetic (49 pairs, 93.9% accuracy)

The arithmetic category tests signed and unsigned overflow semantics. The oracle correctly identifies all UB-related divergences (wrapping vs. UB) and proves equivalence for operations without overflow risk.

Failure cases (3/49): All three failures are complex multi-operation expressions where the parser produces an incomplete IR, causing the oracle to return “unknown.” The verification core is not at fault; extending the parser would resolve these.

Representative pair

```
1 int sat_add(int a, int b) {
2     int sum = a + b;
3     if (a > 0 && b > 0 && sum < 0) return INT_MAX;
4     if (a < 0 && b < 0 && sum > 0) return INT_MIN;
5     return sum;
6 }
```

Listing 11: C: saturating addition.

```
1 pub fn sat_add(a: i32, b: i32) -> i32 {
2     a.saturating_add(b)
3 }
```

Listing 12: Rust: saturating addition.

Oracle verdict: divergent.

Reason: The C function reads `sum` after potential overflow UB; the overflow check itself relies on UB behavior. Rust’s `saturating_add` is well-defined.

D.2 Bitwise (29 pairs, 86.2% accuracy)

Bitwise operations are generally straightforward since both C and Rust define them identically for unsigned types. Divergences arise from:

- Shift operations embedded in bitwise expressions (UB in C)
- Signed right shifts (implementation-defined in C, arithmetic in Rust)
- Complex expressions exceeding parser capabilities

Failure cases (4/29): Two are parser errors on complex expressions; two are incorrect verdicts on signed right-shift pairs where the oracle does not model C’s implementation-defined arithmetic right shift.

D.3 Cast (13 pairs, 100% accuracy)

Type casting is one of the oracle’s strongest categories. C and Rust agree on widening casts and disagree predictably on narrowing casts (C: implementation-defined, Rust: truncation). The σ -bridge encodes this precisely.

D.4 Compound (12 pairs, 100% accuracy)

Compound pairs combine multiple divergence classes (e.g., overflow + shift, division + cast). The oracle correctly handles these by flagging all UB sources independently.

D.5 Control Flow (11 pairs, 100% accuracy)

If/else and ternary expressions are encoded as ITE terms. All 11 pairs are correctly classified. Note that `switch` statements in C are encoded as nested ITE chains.

D.6 Division (21 pairs, 90.5% accuracy)

Division is well-handled, with the oracle correctly identifying division by zero and `INT_MIN / -1` as UB in C.

Failure cases (2/21): Both involve GCD-like loops that exceed the unrolling bound $K = 32$.

D.7 Shift (17 pairs, 100% accuracy)

Shift operations are a key differentiator for SEMREC: C shift by $\geq$ bitwidth is UB, while Rust masks the shift amount. The oracle correctly detects all such divergences.

D.8 Real-World Patterns (26 pairs, 100% accuracy)

These are drawn from real codebases (hash functions, CRC computation, pixel clamping, alignment calculations, ring buffer arithmetic). All use operations within $\mathcal{F}$ and are correctly classified.

Representative pair: FNV-1a hash

```
1 unsigned int fnv1a(unsigned int data) {  
2     unsigned int hash = 2166136261u;  
3     hash ^= data;  
4     hash *= 16777619u;  
5     return hash;  
6 }
```

Listing 13: C: FNV-1a hash (32-bit).

```
1 pub fn fnv1a(data: u32) -> u32 {  
2     let mut hash: u32 = 2166136261;  
3     hash ^= data;  
4     hash = hash.wrapping_mul(16777619);  
5     hash  
6 }
```

Listing 14: Rust: FNV-1a hash (32-bit).

Oracle verdict: equivalent (unsigned arithmetic: no UB in C, wrapping matches).

D.9 Error Handling (8 pairs, 50.0% accuracy)

Error handling patterns are partially supported. Simple error-code returns work; `errno` access and `Result` type conversions require interprocedural analysis and type-level reasoning beyond $\mathcal{F}$.

D.10 Loops (8 pairs, 25.0% accuracy)

Loop support is limited by the unrolling bound $K = 32$. Simple bounded loops with known iteration counts work; unbounded loops (GCD, search) exceed the bound and produce “unknown” verdicts.

D.11 Memory and String (8 pairs, 0% accuracy)

Both categories require pointer operations, heap semantics, or string manipulation—all outside $\mathcal{F}$. The parser correctly rejects these with “error” verdicts rather than producing incorrect results.

E Differential Testing Methodology

E.1 Naive Differential Testing

For each of the 52 core pairs, we compiled both the C and Rust functions, linked them into a shared test harness, and executed both on 10,000 random 32-bit inputs (uniformly distributed). Output comparison used exact equality.

Input generation. For functions with one parameter, we sampled 10,000 random `int32` values. For two-parameter functions, we sampled $100 \times 100 = 10,000$ pairs from the Cartesian product of 100 random values per parameter.

Verdict assignment. If any input produced different outputs, the pair was classified as divergent. If all 10,000 inputs produced identical outputs, the pair was classified as equivalent (no formal guarantee).

E.2 UB-Aware Differential Testing

The UB-aware variant supplements random inputs with explicit edge cases:

- Boundary values: $0, \pm 1, \text{INT_MIN}, \text{INT_MAX}$
- Shift boundaries: $0, 1, 30, 31, 32, 33, -1$
- Division edges: $0, 1, -1, \text{INT_MIN}$
- Type boundaries for each width (e.g., 127, 128, 255 for 8-bit)

This adds approximately 50–200 targeted inputs per pair, depending on the function signature and parameter count.

E.3 Results Breakdown

Table 10: Detailed differential testing results on 52 core pairs.

	Oracle	UB-aware	Naive
True positives (div. detected)	18	16	8
True negatives (equiv. confirmed)	7	10	9
False negatives (div. missed)	11	13	21
False positives (equiv. wrong)	0	0	0
Errors / unknown	16	13	14
Accuracy	48.1%	50.0%	32.7%

The oracle’s lower overall accuracy compared to UB-aware testing reflects parser errors on categories outside $\mathcal{F}$ (strings, memory, complex loops). On the categories within $\mathcal{F}$, the oracle is strictly superior because it can *prove* equivalence and detect UB-invisible divergences.

F CEGAR Experiment Details

F.1 Experimental Setup

- **LLM:** GPT-4.1-nano (accessed via API)
- **Max iterations:** $k = 5$
- **Temperature:** 0.0 (deterministic)
- **Functions:** 30 C functions selected from the core benchmark (excluding string, memory, and complex loop functions outside $\mathcal{F}$)
- **Prompt format:** System prompt + C source + repair context (if applicable)

F.2 Per-Function Results

Table 11: CEGAR per-function results (30 functions, 5 iterations max).

#	Function	Iters	Result	Bug Class
1	cast_u8_to_u32	1	converged	none
2	cast_i16_to_i32	1	converged	none
3	cast_u32_to_u8	1	converged	none
4	reinterpret_sign	1	converged	none
5	cast_chain	1	converged	none
6	add_simple	5	divergent	none (first-try: unknown)
7	add_wrapping	5	divergent	undefined_behavior
8	add_checked	5	divergent	undefined_behavior
9	sub_overflow	5	divergent	undefined_behavior
10	negate_min	5	divergent	undefined_behavior
11	abs_function	5	divergent	undefined_behavior
12	shift_overflow	5	divergent	undefined_behavior
13	mul_simple	5	divergent	other
14	rotate_left	5	divergent	other
15	rotate_right	5	divergent	other
16	bit_extract_wide	5	divergent	other
17	safe_div	5	divergent	other
18	clamp_value	5	divergent	other
19	sign_extend	5	divergent	other
20	pack_bytes	5	divergent	other
21	unpack_byte	5	divergent	other
22	hash_combine	5	divergent	other
23	align_up	5	divergent	none (first-try: error)
24	min_val	5	divergent	none (parser limitation)
25	max_val	5	divergent	none (parser limitation)
26	popcount_manual	5	divergent	none (parser limitation)
27	leading_zeros	5	divergent	none (parser limitation)
28	trailing_zeros	5	divergent	none (parser limitation)
29	byte_swap	5	divergent	none (complex expression)
30	bit_reverse	5	divergent	none (complex expression)

F.3 Naive Retry Baseline

For the naive retry baseline, we selected the 20 functions that produced parseable Rust translations (excluding 10 that caused parser errors). We re-prompted the LLM with only the C source (no counterexample) for 5 fresh attempts per function.

The same 5 cast/reinterpretation functions converged in the naive retry; no additional functions converged. This confirms that the CEGAR feedback adds no marginal value for GPT-4.1-nano on this task.

F.4 Iteration-by-Iteration Analysis

Table 12: CEGAR outcomes by iteration.

	Iter 1	Iter 2	Iter 3	Iter 4	Iter 5
Converged (cumulative)	5	5	5	5	5
New bugs introduced	–	8	6	4	3
Same bug persisted	–	15	17	19	20
Parser error	–	2	2	2	2

The “new bugs introduced” row demonstrates the non-monotonic behavior of neural CEGAR (Remark 6): the LLM often fixes the specific counterexample input but breaks the function on other inputs.

G SMT Encoding Examples

G.1 Signed Addition

For C function `int add(int a, int b) { return a + b; }`:

```

1 (declare-const a (_ BitVec 32))
2 (declare-const b (_ BitVec 32))
3 (declare-const arb_overflow (_ BitVec 32))
4
5 ; Overflow detection for signed addition
6 (define-fun overflow () Bool
7   (or (and (bvsgt a #x00000000) (bvsgt b #x00000000)
8         (bvslt (bvadd a b) #x00000000))
9       (and (bvslt a #x00000000) (bvslt b #x00000000)
10          (bvsgt (bvadd a b) #x00000000))))
11
12 ; C result: unconstrained on overflow (UB)
13 (define-fun c_result () (_ BitVec 32)
14   (ite overflow arb_overflow (bvadd a b)))

```

Listing 15: SMT-LIB encoding for C signed addition.

```

1 ; Rust result: always wrapping (two's complement)
2 (define-fun rust_result () (_ BitVec 32)
3   (bvadd a b))
4
5 ; Product program: check divergence on non-UB inputs
6 (assert (not overflow))
7 (assert (distinct c_result rust_result))
8 (check-sat)

```

Listing 16: SMT-LIB encoding for Rust wrapping addition.

For this pair, the solver returns UNSAT: on all non-overflow inputs, the C and Rust functions agree (both compute `bvadd`). The oracle separately reports the overflow inputs as a UB-class divergence.

G.2 Shift by Width

For C function `int shl(int x, int n) { return x << n; }` and Rust `fn shl(x: i32, n: i32) -> i32 { x.wrapping_shl(n as u32) }`:

```
1 (declare-const x (_ BitVec 32))
2 (declare-const n (_ BitVec 32))
3 (declare-const arb_shift (_ BitVec 32))
4
5 ; C: shift by >= 32 is UB
6 (define-fun shift_ub () Bool
7   (or (bvsge n #x00000020) (bvslt n #x00000000)))
8
9 (define-fun c_result () (_ BitVec 32)
10  (ite shift_ub arb_shift (bvshl x n)))
11
12 ; Rust: mask shift amount to [0, 31]
13 (define-fun rust_result () (_ BitVec 32)
14  (bvshl x (bvand n #x0000001f)))
15
16 (assert (not shift_ub))
17 (assert (distinct c_result rust_result))
18 (check-sat)
```

Listing 17: SMT-LIB encoding for shift comparison.

On non-UB inputs ($0 \leq n < 32$), masking $n \& 31 = n$, so both produce the same result: UNSAT. The oracle reports divergence on inputs with $n \geq 32$ or $n < 0$ as UB-class.

H CLI Reference

SEMREC provides three commands:

H.1 semrec verify

Verify a single C/Rust function pair.

```
# File-based verification
semrec verify --c-file add.c --rs-file add.rs

# Inline verification
semrec verify \
  --c-code 'int_f(int_x){_return_x+1;_}' \
  --rs-code 'pub_fn_f(x:i32)->i32_{x.wrapping_add(1)}'

# With timeout and verbosity
semrec verify --c-file complex.c --rs-file complex.rs \
  --timeout 30 --verbose
```

Output format

```
{
  "verdict": "divergent",
  "counterexample": {
```

```

    "inputs": {"a": 2147483647, "b": 1},
    "c_output": "undefined (signed overflow)",
    "rust_output": -2147483648,
    "divergence_class": "undefined_behavior"
  },
  "repair_hint": {
    "description": "C signed addition has UB on overflow",
    "suggested_fix": "Use wrapping_add or handle overflow explicitly",
    "fix_category": "overflow_semantics"
  },
  "time_seconds": 0.34
}

```

H.2 semrec cegar

Run the CEGAR loop with an LLM.

```

# CEGAR with GPT-4.1-nano, 5 iterations
semrec cegar --source-file overflow.c --model gpt-4.1-nano --max-iter 5

# With custom temperature
semrec cegar --source-file rotate.c --model gpt-4.1-nano \
  --max-iter 10 --temperature 0.2

# Batch CEGAR over a directory
semrec cegar --source-dir benchmarks/core/ --model gpt-4.1-nano \
  --output cegar_results.json

```

H.3 semrec bench

Run the full benchmark suite.

```

# Run all 202 pairs
semrec bench --output results.json

# Run specific category
semrec bench --category arithmetic --output arith_results.json

# Run with verbose per-pair output
semrec bench --verbose --output results.json

```

I Oracle API Reference

I.1 Python API

```

1 from semrec import VerificationOracle
2
3 oracle = VerificationOracle(timeout=10)
4
5 # Verify a pair

```

```

6  result = oracle.verify(
7      c_code='int_add(int_a,int_b){return_a+b;}',
8      rust_code='pub fn add(a:i32,b:i32)->i32{a.wrapping_add(b)}'
9  )
10
11  print(result.verdict)          # "equivalent" | "divergent" | "unknown"
12                                | "error"
13  print(result.counterexample)   # None or CounterExample object
14  print(result.repair_hint)      # None or RepairHint object
15  print(result.time_seconds)     # float
16
17  if result.verdict == "divergent":
18      cex = result.counterexample
19      print(f"Inputs:{cex.inputs}")
20      print(f"C_output:{cex.c_output}")
21      print(f"Rust_output:{cex.rust_output}")
22      print(f"Class:{cex.divergence_class}")
23
24      hint = result.repair_hint
25      print(f"Fix:{hint.description}")
26      print(f"Category:{hint.fix_category}")

```

Listing 18: Full Oracle API usage.

I.2 Data Types

```

1  @dataclass
2  class VerificationResult:
3      verdict: str          # "equivalent" | "divergent" | "unknown" | "
4                             error"
5      counterexample: Optional[CounterExample]
6      repair_hint: Optional[RepairHint]
7      time_seconds: float
8      smt_stats: Optional[SMTStats]
9
10  @dataclass
11  class CounterExample:
12      inputs: Dict[str, int]
13      c_output: Union[int, str] # str for UB description
14      rust_output: Union[int, str]
15      divergence_class: str     # "undefined_behavior" | "
16                               output_mismatch" |
17                               # "type_mismatch" | "panic_divergence"
18
19  @dataclass
20  class RepairHint:
21      description: str
22      suggested_fix: str
23      fix_category: str        # "overflow_semantics" | "shift_masking
24                               " |
25                               # "division_guard" | "cast_truncation"

```

Listing 19: Oracle data types.

J Supported Fragment Formal Definition

Definition 8 (Supported Fragment $\mathcal{F}$). *The supported fragment consists of programs generated by the grammar:*

$$\begin{aligned}
\text{prog} &::= \text{ty id (params) \{ body \}} \\
\text{ty} &::= \text{int} \mid \text{unsigned} \mid \text{char} \mid \text{short} \mid \text{long} \mid \text{i8} \mid \text{i16} \mid \text{i32} \mid \text{i64} \mid \text{u8} \mid \text{u16} \mid \text{u32} \mid \text{u64} \\
\text{params} &::= \varepsilon \mid \text{ty id} \mid \text{ty id, params} \\
\text{body} &::= \text{stmt} \mid \text{stmt body} \\
\text{stmt} &::= \text{ty id} = \text{expr}; \mid \text{return expr}; \mid \text{if (expr) \{body\} else \{body\}} \mid \text{for}(\dots; \text{expr}; \dots) \{body\} \\
\text{expr} &::= \text{id} \mid \text{const} \mid \text{expr binop expr} \mid \text{unop expr} \mid (\text{ty}) \text{expr} \mid \text{expr ? expr : expr} \\
\text{binop} &::= + \mid - \mid \times \mid / \mid \% \mid \& \mid \mid \mid ^ \mid << \mid >> \mid < \mid \leq \mid = \mid \neq \mid \geq \mid > \\
\text{unop} &::= - \mid \sim \mid !
\end{aligned}$$

The Rust variant of this grammar replaces C types with Rust types, C casts with `as` expressions, and adds method-call syntax for `wrapping_*`, `checked_*`, and `saturating_*` operations.

J.1 Excluded Constructs

The following constructs are explicitly excluded from $\mathcal{F}$:

- **Pointers and references:** `*`, `&`, pointer arithmetic, dereferencing
- **Heap allocation:** `malloc`, `free`, `Box`, `Vec`
- **Structs and enums:** aggregate types, field access
- **Strings:** `char*`, `&str`, `String`
- **Arrays:** indexing, bounds checking
- **Function calls:** interprocedural analysis, recursion
- **Floating point:** `float`, `double`, `f32`, `f64`
- **Concurrency:** threads, atomics, synchronization
- **I/O:** `printf`, file operations, `println!`

K Comparison with Related Tools

Table 13: Feature comparison with related tools.

Feature	SemRec	Alive2	Kani	C2Rust	Vert
Cross-language	✓	×	×	✓	✓
Source-level	✓	×	✓	✓	✓
Formal verification	✓	✓	✓	×	×
Detects UB divergence	✓	×	partial	×	×
Counterexample output	✓	✓	✓	×	×
LLM integration	✓	×	×	×	✓
Equivalence proof	✓	✓	×	×	×
Pointer support	×	✓	✓	✓	✓
Heap support	×	✓	✓	✓	partial

SEMREC is unique in combining cross-language, source-level, formal verification with UB-aware divergence detection and LLM integration. Its primary limitation compared to Alive2 and Kani is the lack of pointer and heap support, which restricts the supported fragment to integer-arithmetic functions.

L Threats to Validity

Internal validity. The benchmark was curated by the authors, introducing potential selection bias. We mitigate this by including real-world patterns from established codebases (SQLite, curl, zlib, OpenSSL, Linux kernel, FFmpeg) and by systematically scaling the core benchmark across type variants.

External validity. The supported fragment $\mathcal{F}$ covers only integer-arithmetic functions. Real C-to-Rust migrations involve pointers, heap allocation, and interprocedural dependencies. Our results generalize to the integer-arithmetic subset of migration tasks.

Construct validity. We use GPT-4.1-nano for the CEGAR experiment—a small model. Larger models may achieve higher repair rates. However, the fundamental UB irreparability result (Lemma 7) holds regardless of model capability.

Reproducibility. All experiments use deterministic settings (temperature 0.0) and the benchmark suite is fixed. LLM API behavior may vary over time due to model updates.

M Extended Discussion

M.1 The UB Wall

The most significant finding is the “UB wall”: for functions containing C undefined behavior, no Rust translation can be formally equivalent. This is not a limitation of SEMREC—it is a fundamental property of the C–Rust semantic gap.

Practical implications. Pipeline operators should adopt a three-phase workflow:

1. **Classify:** Use SEMREC to identify which functions have UB divergences vs. clean equivalence.
2. **Auto-verify:** Accept LLM translations for UB-free functions that SEMREC certifies as equivalent.
3. **Manual review:** Flag UB-affected functions for human decision on intended semantics (should overflow wrap? panic? saturate?).

M.2 Why LLMs Fail on Formal Counterexamples

Our negative CEGAR result raises the question: *why* do LLMs fail to apply correct counterexamples? We hypothesize three factors:

1. **Training distribution mismatch.** LLMs are trained on code examples where “fixing a bug” means changing logic or syntax, not reconciling language-level semantic differences.
2. **Implicit semantics.** C’s UB rules are implicit—they are not visible in the source code. The LLM must reason about what the code *does not say* (“this addition may overflow”), which is counter to pattern-matching on code text.

3. **Repair space explosion.** For UB-related bugs, the “correct” repair depends on the *programmer’s intent*, which is not specified. Should `a + b` become `wrapping_add`, `checked_add`, `saturating_add`, or an explicit overflow check? The LLM cannot determine the answer from the code alone.

M.3 Combining Testing and Verification

Our results suggest a complementary strategy: use differential testing as a fast first filter (catches obvious bugs cheaply) and SEMREC as a second pass (catches UB-invisible bugs and proves equivalence for clean translations). The two approaches have complementary strengths:

Table 14: Complementary strengths of testing vs. verification.

Capability	Diff. Testing	SemRec
Fast execution	✓	✓
Proves equivalence	×	✓
Detects UB divergence	×	✓
Handles pointers	✓	×
Handles complex control flow	✓	partial
No false positives	✓	✓
Scales to large functions	✓	partial